ASSIGNMENT- 09

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Advanced SQL Queries Using Joins and Subqueries

By
Name: Mohammed Sinan nm
Reg. No.: 2511021240024

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

ADVANCED SQL QUERIES USING JOINS AND SUBQUERIES

1. Introduction

Advanced SQL queries are used to retrieve and analyze information from multiple related tables in a database. Joins combine related records from different tables, while subqueries allow one query to use the result of another query.

In an Online Food Delivery System, these techniques help generate reports about customer orders, restaurant performance, payment details, delivery status, and customer reviews.

This assignment uses the `FoodDeliveryDB` database created in the previous assignments.

2. Objectives

- To understand and implement different types of SQL joins.
- To retrieve information from multiple related tables.
- To write SQL queries using subqueries.
- To generate meaningful reports for the food delivery system.
- To calculate summary statistics using advanced queries.
- To analyze customer orders, payments, deliveries, and restaurant ratings.

3. Report 1: Customer Order Details Using INNER JOIN

SQL Query

```
USE FoodDeliveryDB;



SELECT

    o.order_id,

    c.full_name AS customer_name,

    r.restaurant_name,

    o.order_date,

    o.total_amount,
```

```
    o.order_status
FROM Orders o
INNER JOIN Customers c
    ON o.customer_id = c.customer_id
INNER JOIN Restaurants r
    ON o.restaurant_id = r.restaurant_id
ORDER BY o.order_id;
```

Sample Output

order_id	customer_name	restaurant_name	total_amount	order_status
1	Mohammed Sinan	Food Palace	320.00	Delivered
2	Rahul Menon	Spice Garden	180.00	Delivered
3	Ananya Nair	Pizza Corner	370.00	Delivered
4	Arjun Kumar	Burger Hub	360.00	Out for Delivery
5	Diya Thomas	Sweet Treats	220.00	Preparing

Note: The sample output omits `order_date` for readability. The actual query also displays that column.

Explanation

The `INNER JOIN` combines the `Orders`, `Customers`, and `Restaurants` tables using their related keys. This report shows which customer placed each order, the restaurant involved, the order amount, and the current order status.

---

4. Report 2: Order Items and Menu Details Using INNER JOIN

SQL Query

```
SELECT
    o.order_id,
    r.restaurant_name,
    m.item_name,
    oi.quantity,
    oi.price_at_order,
    oi.quantity * oi.price_at_order AS item_total
FROM OrderItems oi
INNER JOIN Orders o
    ON oi.order_id = o.order_id
INNER JOIN MenuItems m
    ON oi.item_id = m.item_id
INNER JOIN Restaurants r
    ON m.restaurant_id = r.restaurant_id
ORDER BY o.order_id, m.item_id;
```

Sample Output

order_id	restaurant_name	item_name	quantity	price_at_order	item_total
1	Food Palace	Veg Burger	2	120.00	240.00
1	Food Palace	French Fries	1	80.00	80.00

2	Spice Garden	Masala Dosa	2	90.00	180.00
3	Pizza Corner	Margherita Pizza	1	250.00	250.00
3	Pizza Corner	Garlic Bread	1	120.00	120.00
4	Burger Hub	Chicken Burger	2	180.00	360.00
5	Sweet Treats	Chocolate Cake	1	150.00	150.00
5	Sweet Treats	Ice Cream	1	70.00	70.00

Explanation

This query joins four tables to display the items purchased in each order. The expression `quantity * price_at_order` calculates the total price for each line item using the price recorded when the order was placed.

---

5. Report 3: Customer Order History Using LEFT JOIN

SQL Query

```
SELECT
    c.customer_id,
    c.full_name AS customer_name,
    o.order_id,
    o.total_amount,
    o.order_status
FROM Customers c
```

```
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id
ORDER BY c.customer_id, o.order_id;
```

Sample Output

customer_id	customer_name	order_id	total_amount	order_status
1	Mohammed Sinan	1	320.00	Delivered
2	Rahul Menon	2	180.00	Delivered
3	Ananya Nair	3	370.00	Delivered
4	Arjun Kumar	4	360.00	Out for Delivery
5	Diya Thomas	5	220.00	Preparing

Explanation

A `LEFT JOIN` returns every customer, including customers who have not placed any orders. If a customer has no matching order, the order-related columns appear as `NULL`.

In the sample data, every customer has one order, so all five rows contain order information.

---

6. Report 4: Payment and Order Status Using JOIN

SQL Query

```
SELECT
    o.order_id,
```

```
    c.full_name AS customer_name,
    o.total_amount AS order_amount,
    p.payment_method,
    p.payment_status,
    o.order_status
FROM Orders o
INNER JOIN Customers c
    ON o.customer_id = c.customer_id
INNER JOIN Payments p
    ON o.order_id = p.order_id
ORDER BY o.order_id;
```

Sample Output

order_id	customer_name	order_amount	payment_method	payment_status	order_status
1	Mohammed Sinan	320.00	UPI	Successful	Delivered
2	Rahul Menon	180.00	Card	Successful	Delivered
3	Ananya Nair	370.00	UPI	Successful	Delivered
4	Arjun Kumar	360.00	Cash on Delivery	Pending	Out for Delivery
5	Diya Thomas	220.00	Net Banking	Successful	Preparing

Explanation

This report combines order, customer, and payment information. It helps administrators identify payment methods and check whether payments are successful or pending.

The sample records show that Order 4 has a pending Cash on Delivery payment.

---

7. Report 5: Delivery Assignment Details Using JOIN

SQL Query

```
SELECT
    d.delivery_id,
    o.order_id,
    c.full_name AS customer_name,
    da.agent_name,
    da.phone_number AS agent_phone,
    d.delivery_status
FROM Deliveries d
INNER JOIN Orders o
    ON d.order_id = o.order_id
INNER JOIN Customers c
    ON o.customer_id = c.customer_id
INNER JOIN DeliveryAgents da
    ON d.agent_id = da.agent_id
ORDER BY d.delivery_id;
```

Sample Output

delivery_id	order_id	customer_name	agent_name	agent_phone	delivery_status

1	1	Mohammed Sinan	Arun Kumar	9876543220	Delivered
2	2	Rahul Menon	Vishal Raj	9876543221	Delivered
3	3	Ananya Nair	Kiran Das	9876543222	Delivered
4	4	Arjun Kumar	Rohit Sharma	9876543223	Picked Up

Explanation

This query combines delivery, order, customer, and delivery-agent information. It helps administrators identify the delivery agent assigned to an order and monitor the delivery status.

Order 5 does not appear because no delivery record has been inserted for it yet.

---

8. Report 6: Customers Who Have Placed Orders Using a Subquery

SQL Query

```
SELECT customer_id, full_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
)
ORDER BY customer_id;
```

Sample Output

customer_id	full_name
1	Mohammed Sinan
2	Rahul Menon
3	Ananya Nair
4	Arjun Kumar
5	Diya Thomas

Explanation

The subquery retrieves customer IDs from the `Orders` table. The outer query returns the details of customers whose IDs appear in that result.

This query identifies customers who have placed at least one order.

---

9. Report 7: Orders Above the Average Order Amount Using a Subquery

SQL Query

```
SELECT
    order_id,
    customer_id,
    total_amount,
    order_status
FROM Orders
```

```
WHERE total_amount > (
    SELECT AVG(total_amount)
    FROM Orders
)
ORDER BY total_amount DESC;
```

Sample Output

The average order amount in the sample data is ₹290.

order_id	customer_id	total_amount	order_status
3	3	370.00	Delivered
4	4	360.00	Out for Delivery
1	1	320.00	Delivered

Explanation

The subquery calculates the average amount of all orders. The outer query returns only those orders whose total amount is greater than that average.

This report helps identify orders with higher-than-average values.

---

10. Report 8: Restaurants With Ratings Above the Average Using a Subquery

SQL Query

```
SELECT
    restaurant_id,
```

```
    restaurant_name,
    rating
FROM Restaurants
WHERE rating > (
    SELECT AVG(rating)
    FROM Restaurants
)
ORDER BY rating DESC;
```

Sample Output

The average restaurant rating is **4.32**.

restaurant_id	restaurant_name	rating
3	Pizza Corner	4.6
2	Spice Garden	4.5

Explanation

The subquery calculates the average rating of all restaurants. The outer query displays restaurants whose ratings are higher than the average.

This report helps identify restaurants with above-average ratings.

---

11. Report 9: Restaurant Order Summary Using JOIN and GROUP BY

SQL Query

```
SELECT
```

```
    r.restaurant_name,
    COUNT(o.order_id) AS total_orders,
    SUM(o.total_amount) AS total_order_value,
    AVG(o.total_amount) AS average_order_value
FROM Restaurants r
LEFT JOIN Orders o
    ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY total_order_value DESC;
```

Sample Output

restaurant_name	total_orders	total_order_value	average_order_value
Pizza Corner	1	370.00	370.00
Burger Hub	1	360.00	360.00
Food Palace	1	320.00	320.00
Sweet Treats	1	220.00	220.00
Spice Garden	1	180.00	180.00

Explanation

This query joins the `Restaurants` and `Orders` tables and calculates three summary statistics for each restaurant:

- `COUNT()` counts the number of orders.

- `SUM()` calculates the combined order amount.
- `AVG()` calculates the average order amount.

The `LEFT JOIN` ensures that restaurants without orders are also included in the report. Such restaurants would have a count of zero and `NULL` values for the sum and average.

Note: The total order value represents recorded order amounts, not necessarily completed sales or collected revenue.

---

12. Report 10: Customers Who Have Not Submitted Reviews Using a Subquery

SQL Query

```
SELECT
    c.customer_id,
    c.full_name
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM Reviews r
    WHERE r.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

Sample Output

customer_id	full_name
4	Arjun Kumar

5	Diya Thomas

Explanation

The `NOT EXISTS` subquery checks whether a review exists for each customer. Customers without matching review records are displayed.

In the sample data, customers 4 and 5 have not submitted reviews.

---

13. Report 11: Customers Who Placed Orders Above ₹300 Using a Subquery

SQL Query

```
SELECT
    customer_id,
    full_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    WHERE total_amount > 300
)
ORDER BY customer_id;
```

Sample Output

customer_id	full_name

1	Mohammed Sinan
3	Ananya Nair
4	Arjun Kumar

Explanation

The subquery retrieves the customer IDs associated with orders exceeding ₹300. The outer query uses those IDs to retrieve the corresponding customer names.

This report helps identify customers who placed high-value orders.

---

14. Report 12: Customer Review Summary Using JOIN and Aggregate Functions

SQL Query

```
SELECT
    c.customer_id,
    c.full_name AS customer_name,
    COUNT(r.review_id) AS total_reviews,
    AVG(r.restaurant_rating) AS average_restaurant_rating,
    AVG(r.delivery_rating) AS average_delivery_rating
FROM Customers c
LEFT JOIN Reviews r
    ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.full_name
ORDER BY c.customer_id;
```

Sample Output

The following output assumes these sample review ratings from Assignment 7:

customer_id	customer_name	total_reviews	average_restaurant_rating	average_delivery_rating
1	Mohammed Sinan	1	5.0	4.0
2	Rahul Menon	1	4.0	5.0
3	Ananya Nair	1	5.0	5.0
4	Arjun Kumar	0	NULL	NULL
5	Diya Thomas	0	NULL	NULL

Explanation

This query joins customers with their reviews and calculates the number of reviews and average ratings for each customer. The `LEFT JOIN` includes customers who have not submitted any reviews.

The `NULL` values indicate that no rating data is available for those customers.

---

15. Summary of Advanced SQL Concepts

Concept	Purpose	Example Report
`INNER JOIN`	Retrieves matching records from related tables	Customer order details

`LEFT JOIN`	Includes all records from the left table, even without a match	Customer order history
Multiple-table JOIN	Combines data from three or more related tables	Delivery assignment details
`IN` subquery	Checks whether a value matches a value in a subquery result	Customers who placed orders
Scalar subquery	Returns a single value used by the outer query	Orders above average
`NOT EXISTS`	Identifies records without matching related records	Customers without reviews
`GROUP BY`	Groups records for summary calculations	Restaurant order summary
Aggregate functions	Calculate counts, totals, and averages	Customer review summary

16. Result

Advanced SQL queries were developed using joins, subqueries, and aggregate functions to retrieve and analyze information from the Online Food Delivery System database. The queries generated reports related to customer orders, menu items, payments, delivery agents, restaurant performance, and customer reviews.

17. Conclusion

This assignment demonstrates how joins and subqueries can be used to retrieve meaningful information from multiple related database tables. These techniques help a food delivery platform monitor orders, analyze restaurant performance, track payments and deliveries, and understand customer feedback. Advanced SQL querying is an important skill for database management and data analysis.